



Slot 4: Vom Eigenbau zum CAI-Agenten

Aufgabenblatt für den Workshop der AG Cyberraum

In Slot 3 habt ihr einen Agenten erweitert und auf einer bekannten Aufgabe erprobt. In Slot 4 startet ihr CAI selbst, wählt einen geeigneten CAI-Agenten und untersucht anschließend an einer anspruchsvolleren Aufgabe, welchen zusätzlichen Handlungsraum das Framework eröffnet.

Aufgabe 1: CAI selbst starten und einen CAI-Agenten auswählen

Beendet zuerst den noch aus Slot 1 laufenden Juice-Shop-Container mit Ctrl-C in seinem Terminal. Andernfalls ist der benötigte Port 3000 bereits belegt.

Öffnet anschließend ein neues Terminal und wechselt in die **Wurzel des Teilnehmer-Repositories**. Das ist das Verzeichnis, in dem unter anderem die Ordner Agent und Challenges liegen. Aktualisiert dort den Stand für Slot 4:

```
git pull --ff-only
```

Nach der angekündigten Freigabe müssen dort runtime/compose.yml, runtime/browser_gateway.py und runtime/workshop.defaults.env vorhanden sein. Die öffentliche runtime/workshop.defaults.env enthält Image-Referenzen und Laufzeitlimits. Beim Start liest Compose zusätzlich eure vorhandene Agent/.env für API-Schlüssel, Anbieter und Modell. Ihr müsst den Schlüssel nicht erneut eintragen. Bleibt für alle folgenden Docker-Befehle in der Wurzel des Teilnehmer-Repositories.

Die Images werden über Compose geladen. Dieser Vorbereitungsschritt sollte bereits vor dem Workshop erfolgt sein:

```
docker compose --env-file runtime/workshop.defaults.env -f runtime/compose.yml pull
```

Wurden die Images bereits geladen, könnt ihr direkt starten. Fehlende Images lädt Compose beim Start automatisch nach. Beim Offline-Archiv gilt der von den Lehrenden angegebene lokale Image-Tag in Agent/.env; überspringt dann den Pull. Eine alte runtime/workshop.env wird nicht mehr verwendet.

Startet danach die isolierte Juice-Shop-Instanz, den lokalen Browser-Gateway und den Modell-Gateway zum TU-LiteLLM-Dienst:

```
docker compose --env-file runtime/workshop.defaults.env --env-file Agent/.env -f runtime/compose.yml up -d juice-shop browser-gateway model-gateway
```

Prüft, ob alle drei Dienste gestartet wurden:

```
docker compose --env-file runtime/workshop.defaults.env --env-file Agent/.env -f runtime/compose.yml ps
```

Startet danach die echte CAI-CLI aus dem vorbereiteten Image:

```
docker compose --env-file runtime/workshop.defaults.env --env-file Agent/.env -f runtime/compose.yml run --rm cai
```

Installiert CAI nicht während des Workshops auf eurem Host. Der CAI-Container hat keinen Zugriff auf eure Dateien oder den Docker-Socket. Im Browser erreicht ihr Juice Shop unter <http://127.0.0.1:3000>; innerhalb von CAI lautet die freigegebene Zieladresse <http://juice-shop:3000>.



Untersucht anschließend die Agentenauswahl. Überfliegt die Ausgaben gezielt, statt die vollständigen Prompts zu lesen. Achtet bei jedem Profil nur auf Einsatzzweck, Richtung und verfügbaren Werkzeugraum:

```
/agent list
/agent info blueteam_agent
/agent info bug_bounter_agent
/agent info one_tool_agent
```

Entscheidet erst nach dem Vergleich, welches Profil am besten zum gemeinsamen Juice-Shop-Lauf passt.

1. Warum passt `blueteam_agent` nicht zum vorgesehenen Lauf?

2. Sowohl `bug_bounter_agent` als auch `one_tool_agent` können offensiv arbeiten. Warum passt der als „CTF agent“ angezeigte `one_tool_agent` besser zu dieser fest umrissenen Challenge? Nennt mindestens zwei konkrete Belege aus der Zielbeschreibung und den Profilinformationen.

3. Welche Instruktionen oder Werkzeuge nennt `/agent info` für den ausgewählten Agenten?

Aktiviert anschließend den ausgewählten CAI-Agenten für den gemeinsamen Lauf:

```
/agent select one_tool_agent
```

Alternativ lässt sich derselbe CAI-Agent bereits beim Start explizit auswählen:

```
docker compose --env-file runtime/workshop.defaults.env --env-file Agent/.env -f runtime/
compose.yml run --rm -e CAI_AGENT_TYPE=one_tool_agent cai
```

Aufgabe 2: Technischer Check: Login Admin mit dem CAI-Agenten ausführen

Führt mit dem ausgewählten CAI-Agenten die Login-Admin-Aufgabe aus. Nennt im Auftrag ausdrücklich die freigegebene interne Adresse `http://juice-shop:3000`.

Beobachtet während des Laufs:

- Welche Werkzeugaufrufe und Rückmeldungen werden sichtbar?
- Geht der CAI-Agent genauso wie euer eigener Agent vor?



Aufgabe 3: Product Tampering mit dem Eigenbau-Referenzagenten bearbeiten

Zunächst bearbeitet ihr die Aufgabe **Product Tampering** mit dem Eigenbau-Referenzagenten.

Product Tampering: Verändert in der Produktbeschreibung von **OWASP SSL Advanced Forensic Tool (O-Saft)** das Ziel des „More...“-Links zu <https://owasp.slack.com>. Andere Produktdaten sollen unverändert bleiben. Wenn der verfügbare Aktionsraum dafür nicht ausreicht, erklärt anhand der Ausführungsspur, welcher Schritt fehlt, und beendet den Lauf.

Für den gemeinsamen Vergleich verwendet jedes Team `slot3_reference_agent.py`. Dadurch besitzt der Eigenbau-Referenzagent in allen Teams denselben Aktionsraum aus GET und JSON-POST. Haltet euren eigenen Stand aus Slot 3 für einen zusätzlichen qualitativen Vergleich weiterhin fest.

Die Agenten können sich von Ausführung zu Ausführung unterschiedlich verhalten. Ihr könnt deshalb einen zweiten Lauf mit demselben Zieltext versuchen. Verändert für den gemeinsamen Vergleich weder Code noch System-Prompt des Referenzagenten.

Auch ein nicht gelöstes Ziel ist ein verwertbares Ergebnis, wenn aus der Spur eindeutig hervorgeht, was der Agent als Nächstes tun müsste und warum sein Aktionsraum dafür nicht ausreicht.

Kurze Methodenrecherche: Recherchiert, welche HTTP-Methode typischerweise verwendet wird, um eine vorhandene Ressource unter einer bekannten Adresse vollständig zu ersetzen. Vergleicht das Ergebnis mit den Werkzeugen des Eigenbau-Referenzagenten.

Verwendete Quelle: _____

Recherchierte Methode und Begründung:

Letzte hilfreiche Observation:

Welche konkrete Aktion oder welches Werkzeug fehlt?

Aufgabe 4: Product Tampering mit dem CAI-Agenten bearbeiten

Startet CAI selbst, wählt erneut den als „CTF agent“ angezeigten `one_tool_agent` und gebt ihm wortgleich dasselbe Ziel. Verwendet keine lösungsspezifischen Hinweise aus einem vorherigen Lauf.

Beobachtet insbesondere, wie der Agent das Zielprodukt identifiziert, die bestehende Produktbeschreibung untersucht und eine Änderung der vorhandenen Ressource versucht.

Greift während des Laufs ein, wenn es für die Durchführung erforderlich ist.



Erfolgsbeleg oder Abbruchgrund:

Neues Werkzeug und Zielpfad:

Laufzeit nach Slot 4 beenden

Wenn die Lehrenden das Ende der praktischen CAI-Aufgaben ankündigen, beendet ihr die Laufzeit und entfernt die zugehörigen Container, Netzwerke und temporären Volumes:

```
docker compose --env-file runtime/workshop.defaults.env --env-file Agent/.env -f runtime/
compose.yml down -v
```